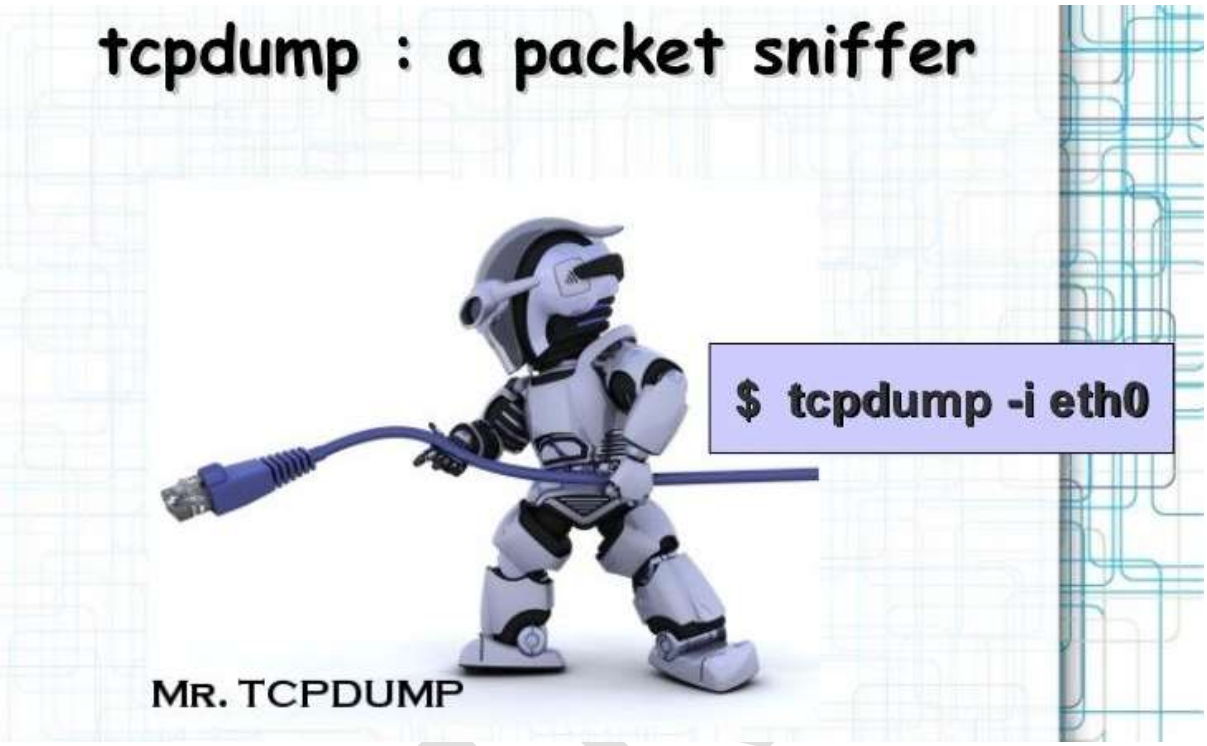
# Tcpdump (Basics)

*Packet Analysis Guide*




Tcpdump is a powerful command-line packet analysis tool used for capturing and inspecting real-time network traffic, enabling users to observe protocol-level communications that are otherwise hidden behind user interfaces. Built on the highly efficient **libpcap** library, Tcpdump allows precise packet capture, advanced filtering (by host, port, protocol, and TCP flags), and flexible output formats including ASCII and hexadecimal views for deep analysis. Its lightweight and fast performance makes it a critical tool for network troubleshooting, security monitoring, and incident analysis.

This documentation and learning approach is strongly inspired by the hands-on, structured content provided by TryHackMe, which played a key role in developing practical packet analysis skills and understanding real network behaviour.

**DOCUMENTED BY**

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

# 1. Overview

**Problem:**

- Network communication is hidden behind UI → hard to understand protocols

**Solution:**

- Capture and analyze real network traffic using **Tcpdump**

**About Tcpdump:**

- CLI-based packet analyzer
- Built on **libpcap**
- Fast, stable, widely used in cybersecurity

# 2. Core Objectives

- Capture network packets
- Apply filters for targeted analysis
- Control packet display formats
- Analyze protocol behavior

WEIRDO

# 3. Basic Packet Capture

**Interface Selection**

- tcpdump -i <interface>
- tcpdump -i any

**Save Capture**

- tcpdump -w file.pcap

**Read Capture**

- tcpdump -r file.pcap

**Limit Packets**

- tcpdump -c <count>

**Disable Name Resolution**

- tcpdump -n # No DNS resolution
- tcpdump -nn # No DNS + port resolution

**Verbose Output**

- tcpdump -v / -vv / -vvv or -v2/ -v3

WEIRDO

# 4. Filtering Packets (Core Concept)

**By Host**

- tcpdump host <IP/hostname>
- tcpdump src host <IP>
- tcpdump dst host <IP>

**By Port**

- tcpdump port <port>
- tcpdump src port <port>
- tcpdump dst port <port>

**By Protocol**

- tcpdump tcp
- tcpdump udp
- tcpdump icmp

**Logical Operators**

```
and     # Both conditions
or      # Either condition
not     # Exclude condition
```

# 5. Advanced Filtering

**Packet Size Filtering**

- greater <size>
- less <size>

**TCP Flags Filtering**

- tcp[tcpflags] == tcp-syn
- tcp[tcpflags] & tcp-syn != 0
- tcp[tcpflags] & (tcp-syn|tcp-ack) != 0

**Concept**

- Uses **binary operations (&, |, !)** on packet header fields
- Enables precise packet-level analysis

WEIRDO

## 6. Display & Output Control

| Option | Description |
|---|---|
| -q | Brief output |
| -e | Show MAC addresses |
| -A | ASCII output |
| -xx | Hex output |
| -X | Hex + ASCII |

## 7. Practical Workflow

**Step-by-Step Usage**

1. **Select Interface**

tcpdump -i eth0

2. **Capture & Save**

tcpdump -i eth0 -w capture.pcap

3. **Apply Filters**

tcpdump -i eth0 tcp port 80

4. **Analyze File**

tcpdump -r capture.pcap -n

5. **Deep Analysis**

tcpdump -r capture.pcap -X

## 8. Key Takeaways

- Tcpdump is essential for **network visibility**
- Filtering is critical → reduces noise
- Works best with .pcap + tools like Wireshark
- Advanced filtering allows **precision-level packet inspection**
- CLI-based → fast and efficient for SOC & pentesting

WEIRDO

## 9. Minimal Cheatsheet

```
tcpdump -i any          # Capture all interfaces
tcpdump -w file.pcap    # Save capture
tcpdump -r file.pcap    # Read capture
tcpdump -n              # No DNS resolution
tcpdump tcp             # Filter TCP
tcpdump port 80         # Filter HTTP
tcpdump host 1.1.1.1    # Filter host
tcpdump -c 10           # Limit packets
tcpdump -X              # Hex + ASCII view
```

## 10. Conclusion

Tcpdump is a powerful packet analysis tool that provides deep visibility into network traffic. It is widely used alongside tools like Wireshark for protocol analysis, threat detection, and troubleshooting. Mastering Tcpdump filtering and output control is crucial for handling large-scale network data efficiently.

WEIRDO